# RAPP v26.11.2 state machine

Stamped, reviewed by the owner

# Contents

1. Purpose . . . 3
2. How to read the tables . . . 4
3. Composition . . . 5
4. Requester remote use (`requester_remote_use`) . . . 6
5. Requester view of the custodian (`requester_view_of_custodian`) . . . 9
6. Custodian service (`custodian_service`) . . . 12
7. The custodian's view of each computer . . . 15
8. Pairing . . . 16
9. Session, close reasons and the rules between machines . . . 17
   9.1 Session . . . 17
   9.2 What a received close does . . . 17
   9.3 Cross-component rules . . . 17
   9.4 Lifecycle invariants . . . 18
10. Roles and names . . . 19
11. Known limits . . . 20
12. Analysis . . . 21

**Status:** Stamped, reviewed by the owner
**Applies to:** RAPP v26.11.2, the transport and discovery hierarchy, the formal state machine and the lifecycle scenarios

---

# 1. Purpose

This document explains the lifecycle of a custodian (the phone that serves its card) and a requester (the computer or tablet that uses it) as one set of state machines, in the style of a TCP state diagram, so that every combination of state and event has a stated answer. It explains the formal state machine: the tables below are generated from it, and where the two differ the formal machine is right.

The formal state machine defines six machines. The pairing, session and operation machines describe one pairing, one connection and one operation. The three lifecycle machines above them describe what holds those instances together: the requester's remote-use switch, its view of whether the custodian can be reached, and the custodian's service. How the custodian's holder sees each computer is the pairing machine itself (section 7). The operation machine is outside this document.

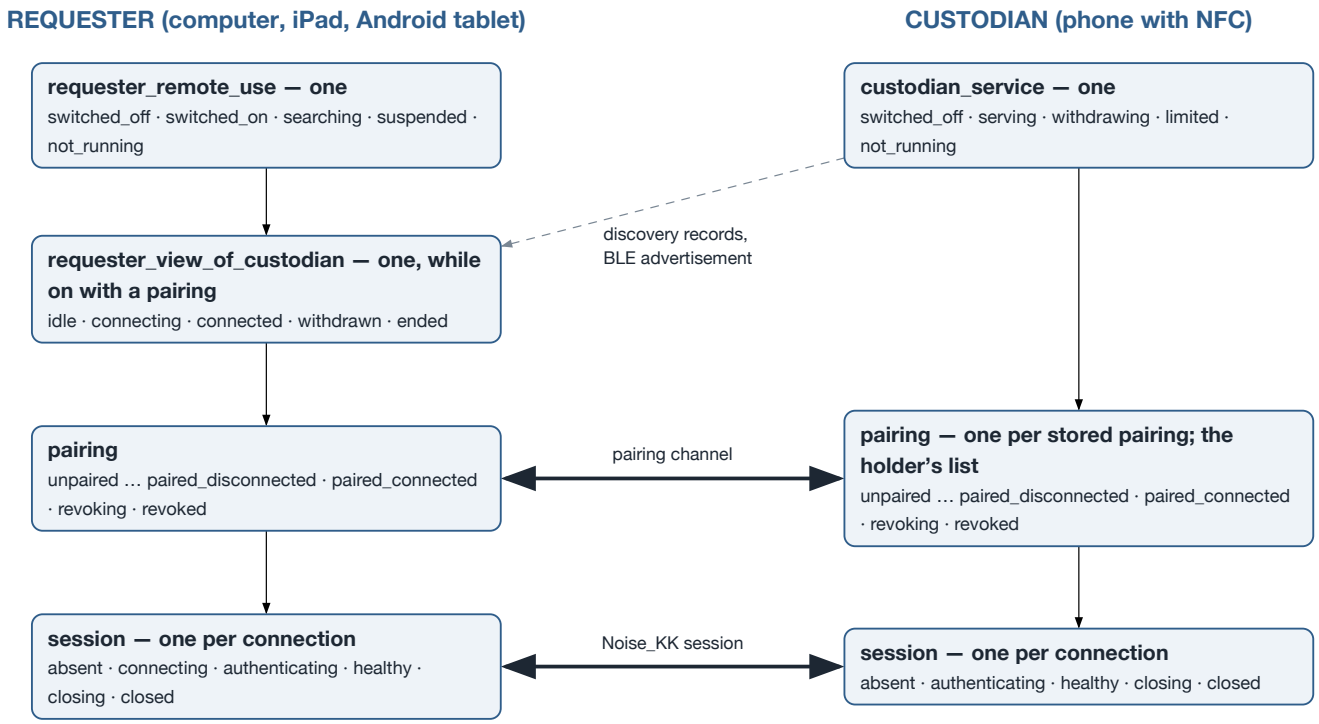
# 2. How to read the tables

Each lifecycle machine has a table with one row per event and one column per state. A cell holds one of:

| Cell | Meaning |
|---|---|
| `state` | A transition to that state; its ordered actions are listed below the table. |
| `state` if *guard* | One of several transitions told apart by guards that between them cover every case. |
| — | Ignored: the event changes nothing in that state. The reason is listed below the table. |
| × *letter* | Cannot happen; the letter names the reason, listed below the table. A guarded × cannot happen while its guard holds. |

Guards and actions are the names the formal machine registers under `guards` and `actions`; their meaning is stated there in one line each. The lifecycle scenarios walk these machines step by step and are checked against the formal machine.

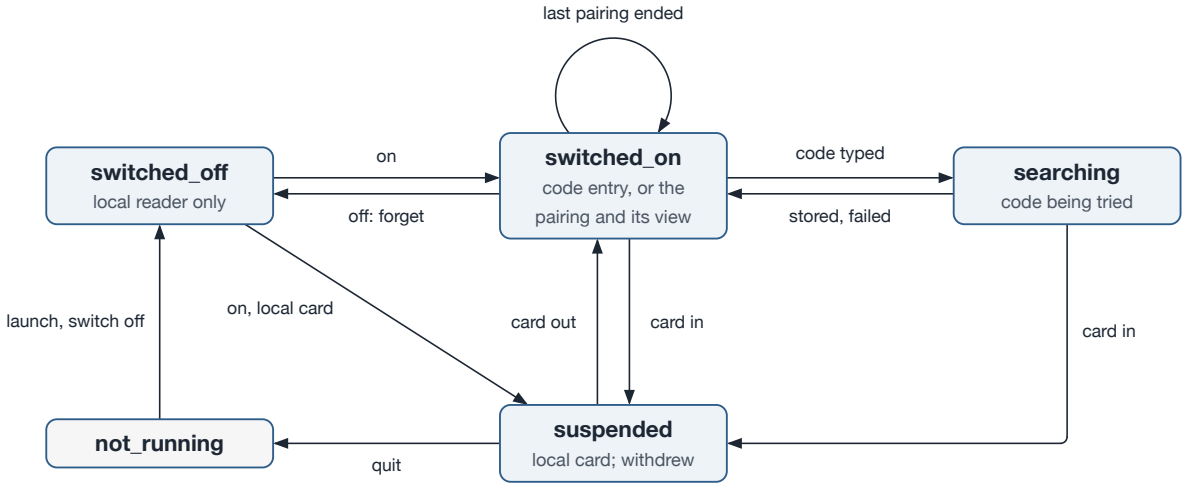
# 3. Composition




Arrows down are creation and control; the machines talk upwards through the events of section 9. The two pairing machines and the two session machines talk to each other only through the wire (double lines). The requester learns of the custodian's service only through discovery records, the BLE advertisement and its session's close reasons; the custodian learns of the requester's remote use only through sessions and their close reasons.

A requester holds at most one pairing (INV-17), so it runs at most one `requester_view_of_custodian`, and only while its remote use is `switched_on` with a pairing.

# 4. Requester remote use (`requester_remote_use`)




*off: forget* also leaves `searching` and `suspended`, each putting the pairing into the revocation backlog; a local card present does not stop it. *quit* leaves every running state for `not_running`; from `switched_on` it first sends `service_withdrawn`, waiting at most 1 s. A launch restores the stored switch: `switched_on` or `suspended` as the local card says, `switched_off` otherwise.

| State | Remote activity | Pairing | User sees |
|---|---|---|---|
| `switched_off` | none, apart from the revocation backlog of pairings its user forgot | none active | the switch off; the local reader only |
| `switched_on` | with a pairing: browses `mode=session` and scans BLE continuously; one `requester_view_of_custodian` | none or one active | the remote card while one is presented; otherwise no remote card and the code entry |
| `searching` | browses `mode=pairing` and tries the custodians found (RAPP §4.2) | the old one, if any, until the new one is stored | the code being tried |
| `suspended` | none: it withdrew (RAPP §4.5 item 8) | kept | the local card |
| `not_running` | none | kept | nothing |

| Event | `switched_off` | `switched_on` | `searching` | `suspended` | `not_running` |
|---|---|---|---|---|---|
| `holder_turns_on` | `switched_on` if no_local_card; `suspended` if local_card_present | × a | × a | × a | × i |
| `holder_turns_off` | × b | `switched_off` | `switched_off` | `switched_off` | × i |
| `code_typed` | × c | `searching` if no_remote_card_presented; × d if remote_card_presented | × c | × c | × i |
| `pairing_stored` | × e | × e | `switched_on` | × e | × i |
| `pairing_failed` | × e | × e | `switched_on` if holds_active_pairing; `switched_on` if holds_no_active_pairing | × e | × i |
| `local_card_inserted` | — | `suspended` | `suspended` | — | × i |
| `local_card_removed` | — | × f | × f | `switched_on` if holds_active_pairing; `switched_on` if holds_no_active_pairing | × i |
| `holder_quits` | `not_running` | `not_running` | `not_running` | `not_running` | × i |
| `launched` | × g | × g | × g | × g | `switched_off` if remote_use_switch_off; `suspended` if remote_use_switch_on, local_card_present; `switched_on` if remote_use_ |

| Event | switched_off | switched_on | searching | suspended | not_running |
|---|---|---|---|---|---|
| | | | | | switch_on, no_local_card, holds_active_pairing; switched_on if remote_use_switch_on, no_local_card, holds_no_active_pairing |
| process_ended | not_running | not_running | not_running | not_running | × i |
| last_pairing_ended | × h | switched_on | — | — | × i |
| sleep_or_wake | — | — | — | — | × i |

Why cells cannot happen:

- **a**: the switch is already on.
- **b**: the switch is already off.
- **c**: the code entry is offered only in switched_on, while no remote card is presented (hierarchy specification section 5.2).
- **d**: the code entry is not offered while a remote card is presented.
- **e**: no pairing search runs; a local card ends one with search_interrupted.
- **f**: no local card is present in that state; one arriving moves to suspended.
- **g**: a running process does not launch.
- **h**: no active pairing is held while remote use is off.
- **i**: no process runs to take the event; launched reads the stored switch, the local reader and the pairing afresh.

Transitions and their ordered actions:

| From | Event | Guard | To | Actions |
|---|---|---|---|---|
| switched_off | holder_turns_on | no_local_card | switched_on | request_remote_permissions, show_code_entry |
| switched_off | holder_turns_on | local_card_present | suspended | request_remote_permissions |
| switched_off, searching, suspended | holder_quits | | not_running | none |
| switched_on | holder_quits | | not_running | withdraw_requester_within_quit_bound |
| switched_off, switched_on, searching, suspended | process_ended | | not_running | none |
| switched_on | holder_turns_off | | switched_off | forget_held_pairing, hide_code_entry, stop_session_discovery |
| switched_on | code_typed | no_remote_card_presented | searching | hide_code_entry, stop_session_discovery, start_pairing_search |
| switched_on | local_card_inserted | | suspended | hide_code_entry, withdraw_requester, stop_session_discovery |
| switched_on | last_pairing_ended | | switched_on | stop_session_discovery, show_code_entry |
| searching | holder_turns_off | | switched_off | end_pairing_search, forget_held_pairing |
| searching | pairing_stored | | switched_on | end_pairing_search, forget_previous_pairing, start_session_discovery |
| searching | pairing_failed | holds_active_pairing | switched_on | end_pairing_search, start_session_discovery, show_code_entry |
| searching | pairing_failed | holds_no_active_pairing | switched_on | end_pairing_search, show_code_entry |
| searching | local_card_inserted | | suspended | end_pairing_search |
| suspended | holder_turns_off | | switched_off | forget_held_pairing |
| suspended | local_card_removed | holds_active_pairing | switched_on | start_session_discovery, show_code_entry |
| suspended | local_card_removed | holds_no_active_pairing | switched_on | show_code_entry |
| not_running | launched | remote_use_switch_off | switched_off | none |
| not_running | launched | remote_use_switch_on, local_card_present | suspended | none |
| not_running | launched | remote_use_switch_on, no_local_card, holds_active_pairing | switched_on | start_session_discovery, show_code_entry |

| From | Event | Guard | To | Actions |
|---|---|---|---|---|
| not_running | launched | remote_use_switch_on, no_local_card, holds_no_active_pairing | switched_on | show_code_entry |

Ignored events:

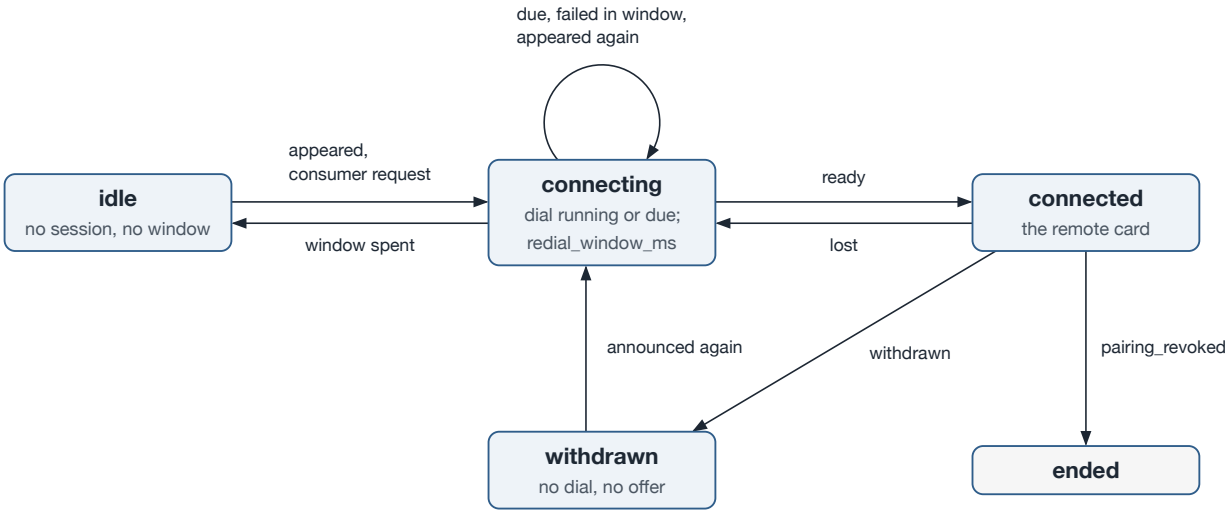
- switched_off / local_card_inserted, local_card_removed: remote use is off; the local reader works as it always does.
- switched_off, switched_on, searching, suspended / sleep_or_wake: sleep is not withdrawal (section 4.5); a session that dies in sleep is a loss to requester_view_of_custodian, and a search it breaks ends as pairing_failed.
- searching / last_pairing_ended: the pairing being replaced ended; the search goes on and its result decides through holds_active_pairing.
- suspended / local_card_inserted: remote use is already suspended.
- suspended / last_pairing_ended: the card's removal finds no active pairing and offers the code entry.

Notes:

- **Switch.** Only the holder's click moves the switch. The requester stores its position and restores it at launch: launched is raised after sweep_stored_pairings (unconfirmed records revoked, a revoking pairing's key past its bound destroyed, idle expiry run), so its guards read the swept pairings. The daily idle-expiry sweep runs in every running state. A revocation from the custodian or idle expiry leaves the switch on and shows the code entry (switched_on / last_pairing_ended).
- **One state for on.** Whether the requester holds a pairing is the guard holds_active_pairing, not a state. The code entry is shown in switched_on exactly while no_remote_card_presented holds, that is while no identity is offered, a running grace included.
- **Off is forget.** holder_turns_off raises forget_pairing for the held pairing, which goes to revoking and joins the requester's revocation backlog (section 8, RAPP §4.6 item 6). With a session open the close goes out at once and the requester waits at most quit_withdrawal_bound_ms for its echo. With none, or without the echo, the requester keeps browsing for the pairing's discovery hint and scanning for the RAPP BLE advertisement, and each time the custodian appears it opens one connection that sends the close as the session's first message. Every forget the user starts does this: switching remote use off, with or without a local card present, and replacing the pairing by pairing another phone. The forgotten pairing offers no identity and starts no operation; it goes on the custodian's echo, or when its idle expiry would have passed, pairing_idle_expiry_days after its latest established session. Several forgotten pairings may wait at once. The backlog is the only exception to CC-86: a local card present does not stop it. Automatic drops send nothing: idle expiry goes straight to revoked on the requester (section 8), and a local card only suspends remote use and keeps the pairing.
- **One pairing.** Typing a code stops session discovery, which ends the view of the custodian; a failed search with the pairing still held starts it again. A pairing stored in searching replaces the one held (forget_previous_pairing, X-10): the old pairing is forgotten as at switch-off, into the revocation backlog.
- **Discovery and the view.** start_session_discovery starts a requester_view_of_custodian in idle; stop_session_discovery ends it with pairing_left. The revocation backlog keeps the discovery it needs while a forgotten pairing waits in it.
- **Permissions.** holder_turns_on is the only moment the requester asks the operating system for local-network and Bluetooth permission; launched asks for none.
- **Sleep.** Sleep is not withdrawal (RAPP §4.5). A session that dies in sleep is a loss to requester_view_of_custodian.

# 5. Requester view of the custodian (`requester_view_of_custodian`)

The requester's process model of its paired custodian: whether a session is up, and whether to dial. One instance runs while remote use is `switched_on` with a pairing; it ends when the pairing stops being active or discovery stops.




*lost*: a loss, or a close with `normal`, `complete`, `user_disconnect`, `policy` or `shutdown`; it opens the redial window at once. *window spent*: the window ended with no dial running, or a dial failed after it; a protocol violation also returns to `idle`, without redial. *withdrawn*: `service_withdrawn` or `card_unavailable` received; a withdrawn record matched in `idle` or `connecting` leads there too.
*announced again*: a `mode=session` record, or the BLE advertisement first seen after the withdrawal. A consumer request in `withdrawn` is refused. `pairing_left` ends every state.

| State | Session | Dials | User sees |
|---|---|---|---|
| `idle` | none | when the custodian appears or a consumer asks | no remote card, or the identity still under a running grace |
| `connecting` | being set up, or due on the back-off | within `redial_window_ms`, at once on an appearance or a consumer request | no remote card, or the identity still under a running grace |
| `connected` | healthy | no | the remote card, once a session confirmed it |
| `withdrawn` | none | never, until the custodian announces itself | no remote card |
| `ended` | none | never | nothing of this pairing; after the custodian's `pairing_revoked`, the line that the phone forgot this computer |

**Presentation.** The requester shows the custodian only through its card. Whether the custodian is out of sight, withdrawn, lost or being dialed, the requester shows no remote card, exactly as if no phone were there, and names neither the custodian's state nor a reason. What it shows after a loss comes only from the grace of RAPP §8.4 item 3. Dialing, scanning and redialing are background behaviour bounded by the redial window and are never shown. The one line that names the custodian is the notice, after its `pairing_revoked`, that the phone forgot this computer. In every state the user's actions are the ordinary ones: use a card in a reader attached to the computer, pair another phone (which replaces the pairing), or turn remote use off.

**How a session end is treated** (RAPP §4.5 item 6):

| What ended it | Next state | Identity |
|---|---|---|
| `pairing_revoked` received | `ended`; the pairing is forgotten | withdrawn at once |
| `service_withdrawn` received, or a withdrawn record verified | `withdrawn` | withdrawn at once |
| `card_unavailable` received | `withdrawn`; no redial, the requester waits for the custodian's next announcement | withdrawn at once, with no grace: a certificate whose key cannot be used is never offered |
| `protocol_violation`, received or detected by the requester | `idle`; no redial: the requester dials again only when the custodian next appears or a consumer asks | offered under the grace of RAPP §8.4 item 3 |
| `normal`, `complete`, `user_disconnect`, `policy` or `shutdown` received, or a loss | `connecting`, with the redial window opened at once | offered under the grace of RAPP §8.4 item 3 |

Only an explicit withdrawal, `card_unavailable`, a revocation and a protocol violation stop the redialing; the pairing is kept in each case but revocation. After `card_unavailable` the custodian announces itself afresh when a card is back

(section 6), and that appearance dials. A protocol violation most likely means the two endpoints run different versions, which redialing would not cure.

**The redial window.** `redial_window_ms` (120 000) equals `identity_offer_grace_ms`. A loss opens it at once, whatever is in sight, and so does the first failed dial since the custodian last appeared; within it the requester redials on its own back-off and keeps offering the identity over one and the same window, so it stops redialing when it stops offering. When the window ends with no dial running the view goes back to `idle`; a dial still running finishes first. Outside the window the requester dials only when the custodian appears or a consumer asks (X-07); nothing dials on a free-running timer. Whether the custodian is in sight decides no cell: `sight_lost` changes nothing in `idle` and `connecting`.

| Event | idle | connecting | connected | withdrawn | ended |
|---|---|---|---|---|---|
| record_appeared | connecting | connecting | — | connecting | × f |
| advertisement_appeared | connecting | connecting | — | connecting | × f |
| withdrawn_record_matched | withdrawn | withdrawn | — | — | × f |
| sight_lost | — | — | — | — | × f |
| consumer_request | connecting | connecting | × a | withdrawn | × f |
| redial_due | × b | connecting | × b | × b | × f |
| redial_window_ended | × b | idle if no_dial_running; connecting if dial_running | × b | × b | × f |
| dial_succeeded | × c | connected | × c | × c | × f |
| dial_failed | × c | connecting if within_redial_window; idle if redial_window_spent | × c | × c | × f |
| close_service_withdrawn | × d | × e | withdrawn | × d | × f |
| close_pairing_revoked | × d | ended | ended | × d | × f |
| close_card_unavailable | × d | × e | withdrawn | × d | × f |
| close_other_reason | × d | × e | connecting | × d | × f |
| session_protocol_violation | × d | idle | idle | × d | × f |
| session_lost | × d | × e | connecting | × d | × f |
| pairing_left | ended | ended | ended | ended | × f |

Why cells cannot happen:

- **a**: a session is open; the request goes on it.
- **b**: no redial window runs in that state.
- **c**: no dial is being set up.
- **d**: no session or dial exists to carry it.
- **e**: these follow session.ready; before it a custodian's revoking close is close_pairing_revoked and anything else ends the dial as dial_failed.
- **f**: the instance has ended; discovery started later starts a new one in idle.

Transitions and their ordered actions:

| From | Event | Guard | To | Actions |
|---|---|---|---|---|
| idle, withdrawn | record_appeared, advertisement_appeared | | connecting | open_session_dial |
| connecting | record_appeared | | connecting | abort_dial, open_session_dial |
| connecting | advertisement_appeared | | connecting | open_session_dial |
| idle | withdrawn_record_matched | | withdrawn | withdraw_identity |
| connecting | withdrawn_record_matched | | withdrawn | abort_dial, end_redial_window, withdraw_identity |
| idle, connecting | consumer_request | | connecting | open_session_dial |
| withdrawn | consumer_request | | withdrawn | refuse_consumer_request |
| connecting | redial_due | | connecting | open_session_dial |
| connecting | redial_window_ended | no_dial_running | idle | end_redial_window |
| connecting | redial_window_ended | dial_running | connecting | none |
| connecting | dial_succeeded | | connected | end_redial_window |
| connecting | dial_failed | within_redial_window | connecting | schedule_redial |
| connecting | dial_failed | redial_window_spent | idle | end_redial_window |
| connected | close_service_withdrawn, close_card_unavailable | | withdrawn | withdraw_identity |

| From | Event | Guard | To | Actions |
|---|---|---|---|---|
| `connecting`, `connected` | `close_pairing_revoked` | | `ended` | `withdraw_identity`, `show_peer_forgot` |
| `connected` | `close_other_reason`, `session_lost` | | `connecting` | `start_offer_grace`, `schedule_redial` |
| `connected` | `session_protocol_violation` | | `idle` | `start_offer_grace` |
| `connecting` | `session_protocol_violation` | | `idle` | `abort_dial`, `end_redial_window` |
| `idle`, `withdrawn` | `pairing_left` | | `ended` | `withdraw_identity` |
| `connecting` | `pairing_left` | | `ended` | `abort_dial`, `end_redial_window`, `withdraw_identity` |
| `connected` | `pairing_left` | | `ended` | `withdraw_identity`, `request_local_close` |

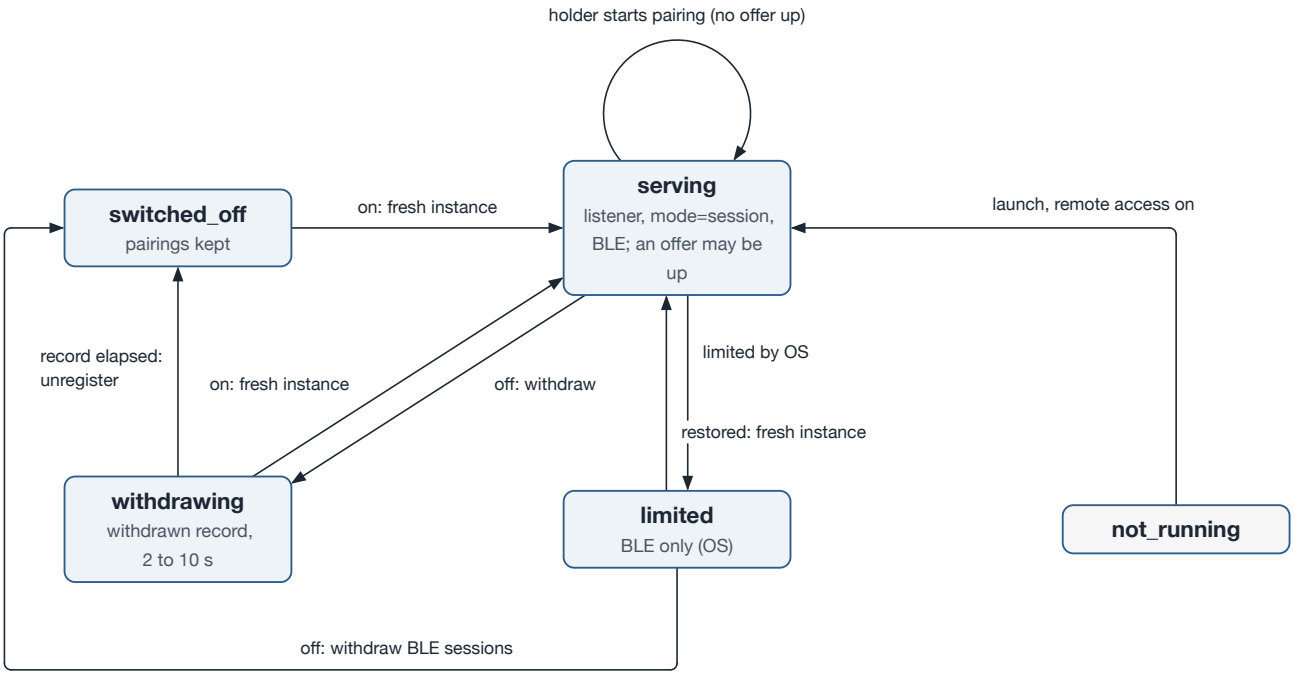
Ignored events:

- `idle` / `sight_lost`: idle waits for an appearance or a consumer whatever is in sight.
- `connecting` / `sight_lost`: the redial window runs on; its dials go to the remembered endpoint or a BLE scan.
- `connected` / `record_appeared`, `advertisement_appeared`, `withdrawn_record_matched`, `sight_lost`: an open session outranks discovery (section 4.5 item 6).
- `withdrawn` / `withdrawn_record_matched`, `sight_lost`: already withdrawn; the withdrawn record's unregistration announces nothing.

Notes:

- **Appearance.** `record_appeared` is an edge: first sight, a fresh instance name, or back after it was lost from sight. A `mode=session` record without hints counts for the pairing; routing decides. A record that appears while a dial is running (a BLE scan, say) aborts it and dials the stream at once (hierarchy §3).
- **Withdrawn.** A verified withdrawal withdraws the identity, drops a dial being set up and ends the redial window. The withdrawn record's later unregistration announces nothing. A BLE scan in `withdrawn` is passive; only an advertisement first seen after the withdrawal moves to `connecting`. A consumer that asks is answered at once that no remote card is available (X-06, INV-16).
- **Open session first.** In `connected` discovery records change nothing (RAPP §4.5 item 6).

# 6. Custodian service (`custodian_service`)




Withdraw: cancel any offer, then `service_withdrawn` on every session, stop accepting, withdrawn TXT record in place, then unregister after 2 to 10 s (RAPP §4.5 item 3).
Being limited also cancels an offer; quitting withdraws as far as the platform lets it. A card available again after `card_unavailable` re-announces `serving` under a fresh instance name.

| State | Listener and records | Sessions | Pairings |
|---|---|---|---|
| `switched_off` | none | none | kept |
| `serving` | stream listener; `mode=session` record; BLE advertisement; while an offer is up, also its `mode=pairing` record | as many as requesters dial, kept and routed while an offer is up | kept |
| `withdrawing` | withdrawn record, accepting nothing, for 2 to 10 s | closed with `service_withdrawn` | kept |
| `limited` | BLE only; no stream listener and no record | BLE sessions | kept |
| `not_running` | none | none | kept |

| Event | `switched_off` | `serving` | `withdrawing` | `limited` | `not_running` |
|---|---|---|---|---|---|
| `holder_turns_on` | `serving` | × a | `serving` | × a | × j |
| `holder_turns_off` | × b | `withdrawing` | × b | `switched_off` | × j |
| `holder_starts_pairing` | × c | `serving` if no_offer_up; × d if offer_up | × c | × c | × j |
| `offer_ended` | × e | — | × e | × e | × j |
| `serving_limited_by_os` | — | `limited` | — | × f | × j |
| `serving_restored_by_os` | — | × g | — | `serving` | × j |
| `holder_quits` | `not_running` | `not_running` | `not_running` | `not_running` | × j |
| `launched` | × h | × h | × h | × h | `serving` if remote_access_on; `switched_off` if remote_access_off |
| `process_ended` | `not_running` | `not_running` | `not_running` | `not_running` | × j |
| `withdrawn_record_elapsed` | × i | × i | `switched_off` | × i | × j |
| `network_changed` | — | `serving` | — | — | × j |
| `hint_epoch_rolled` | — | `serving` | — | — | × j |
| `card_available_again` | — | `serving` | — | — | × j |

Why cells cannot happen:

- **a**: remote access is already on.
- **b**: remote access is already off.
- **c**: the pairing control is offered only while the custodian serves in the foreground.
- **d**: an offer is already up.

- **e**: no offer exists; leaving serving cancels it.
- **f**: already limited.
- **g**: not limited.
- **h**: a running process does not launch.
- **i**: the withdrawn record runs only in withdrawing; turning remote access on there unregisters the withdrawn instance and cancels its timer.
- **j**: no process runs to take the event.

Transitions and their ordered actions:

| From | Event | Guard | To | Actions |
|---|---|---|---|---|
| `serving` | `card_available_again` | | `serving` | `reannounce_service` |
| `switched_off` | `holder_turns_on` | | `serving` | `register_fresh_instance`, `start_ble_advertising` |
| `withdrawing` | `holder_turns_on` | | `serving` | `unregister_service`, `register_fresh_instance`, `start_ble_advertising` |
| `serving` | `holder_turns_off` | | `withdrawing` | `cancel_offer`, `withdraw_custodian`, `stop_ble_advertising` |
| `limited` | `holder_turns_off` | | `switched_off` | `withdraw_custodian_sessions`, `stop_ble_advertising` |
| `serving` | `holder_starts_pairing` | `no_offer_up` | `serving` | none |
| `serving` | `serving_limited_by_os` | | `limited` | `cancel_offer` |
| `limited` | `serving_restored_by_os` | | `serving` | `register_fresh_instance` |
| `serving`, `limited` | `holder_quits` | | `not_running` | `withdraw_custodian_best_effort` |
| `switched_off`, `withdrawing` | `holder_quits` | | `not_running` | none |
| `switched_off`, `serving`, `withdrawing`, `limited` | `process_ended` | | `not_running` | none |
| `withdrawing` | `withdrawn_record_elapsed` | | `switched_off` | `unregister_service` |
| `serving` | `network_changed` | | `serving` | `republish_on_current_interfaces` |
| `serving` | `hint_epoch_rolled` | | `serving` | `update_txt_in_place` |
| `not_running` | `launched` | `remote_access_on` | `serving` | `register_fresh_instance`, `start_ble_advertising` |
| `not_running` | `launched` | `remote_access_off` | `switched_off` | none |

Ignored events:

- `serving` / `offer_ended`: the pairing machine ends the offer; the service is unchanged.
- `switched_off` / `serving_limited_by_os`, `serving_restored_by_os`, `network_changed`, `hint_epoch_rolled`: nothing is served.
- `withdrawing` / `serving_limited_by_os`, `serving_restored_by_os`, `network_changed`, `hint_epoch_rolled`: the withdrawal runs to its end as far as the platform lets it.
- `limited` / `network_changed`, `hint_epoch_rolled`: no record is published while limited; BLE carries no hint.
- `switched_off`, `withdrawing`, `limited` / `card_available_again`: nothing to announce now; serving again registers a fresh instance anyway.

Notes:

- **Offering a pairing is not a state.** `holder_starts_pairing` in `serving` starts the pairing machine's offer and leaves the service as it was; the guard `no_offer_up` admits it and `offer_up` makes it impossible. The offer ends in the pairing machine, so `offer_ended` changes nothing here. Turning remote access off and being limited by the operating system cancel any open offer.
- **Off is withdrawal.** Turning remote access off runs RAPP §4.5 item 3 in order and touches no pairing (X-08, INV-14). Turning it on, also while a withdrawal is still being announced, serves at once under a fresh instance name with every pairing kept.
- **Records in place.** The withdrawn record and the discovery hints of each new 15-minute epoch replace the TXT record of the current instance in place; the instance stays registered and keeps its name.
- **On during a withdrawal.** Turning remote access on in `withdrawing` unregisters the withdrawn instance at once and registers a fresh one.
- **Off while limited.** With no instance registered, turning remote access off closes the sessions with `service_withdrawn` and stops accepting; there is no record to replace (`withdraw_custodian_sessions`).
- **Card back.** After closing its sessions with `card_unavailable`, a custodian that can serve a card again announces itself afresh (`card_available_again` → `reannounce_service`): a fresh instance name and a restarted BLE

advertisement, so every requester sees an appearance and dials. The event changes nothing in `switched_off`, `withdrawing` and `limited`, where serving again registers a fresh instance anyway.

- **Limited.** An operating system that takes the stream listener away (an iPhone in the background) ends the stream sessions without a message; each requester sees a loss. This is not withdrawal.

# 7. The custodian's view of each computer

The custodian's holder sees each paired computer through the pairing machine itself, under the display name stored in the pair record. Its `presentation` map in the formal machine says what the list shows:

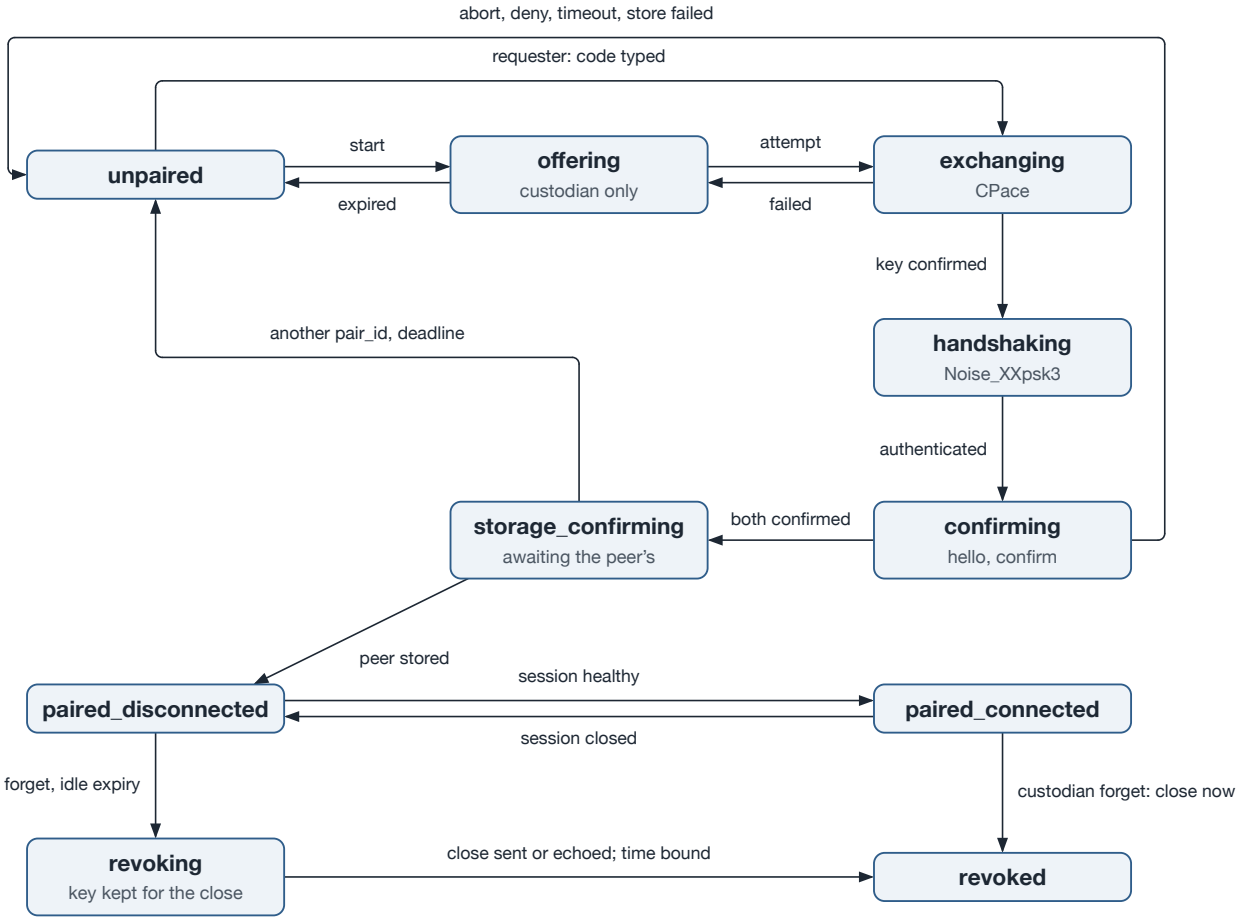
| Pairing state | The holder's list |
|---|---|
| `paired_connected` | listed under its stored display name, connected |
| `paired_disconnected` | listed, not connected; after the computer's `service_withdrawn`, off until the pairing's next session (`show_peer_withdrawn`, RAPP §4.5 item 9) |
| `revoking` | not listed; the key is kept for the close |
| `revoked` | not listed |

The computer's `service_withdrawn` arrives as `peer_withdrawal_notice`, its `pairing_revoked` as `peer_revocation_notice`, and the holder forgetting the computer is `forget_pairing`. Re-pairing the same computer makes a new, unrelated pairing (section 11).

# 8. Pairing

The `pairing` machine. Two of its paths matter most for the lifecycle:

- **Forget.** On a custodian `forget_pairing` from `paired_connected` sends the close at once and destroys the key; from `paired_disconnected` it goes to `revoking`, and the key is kept for the close for at most `revocation_notice_days` (`notice_window_expired`). On a requester `forget_pairing` goes to `revoking` from either state, sending the close at once from `paired_connected`, and the pairing waits in the revocation backlog: `revocation_notice_sent` when the custodian echoed the close, or `notice_window_expired` when the pairing's idle expiry would have passed, `pairing_idle_expiry_days` after its latest established session. A close without its echo leaves it `revoking` for the custodian's next appearance (X-09, X-12, INV-18).
- **Idle expiry by role.** On a custodian `idle_expired` goes to `revoking`, and the close goes out on the next session within the notice window. On a requester it goes straight to `revoked`: the record is marked revoked, the key destroyed and the identity withdrawn, and nothing is sent.
- **Store failure.** `pair_record_store_failed` from `confirming`: nothing is stored, no `pairing.stored` is sent, `pairing.abort` goes out, the candidate keys are destroyed and the user is told the pairing could not be stored, with the platform's cause. A peer that stored its record revokes it at its confirmation deadline.



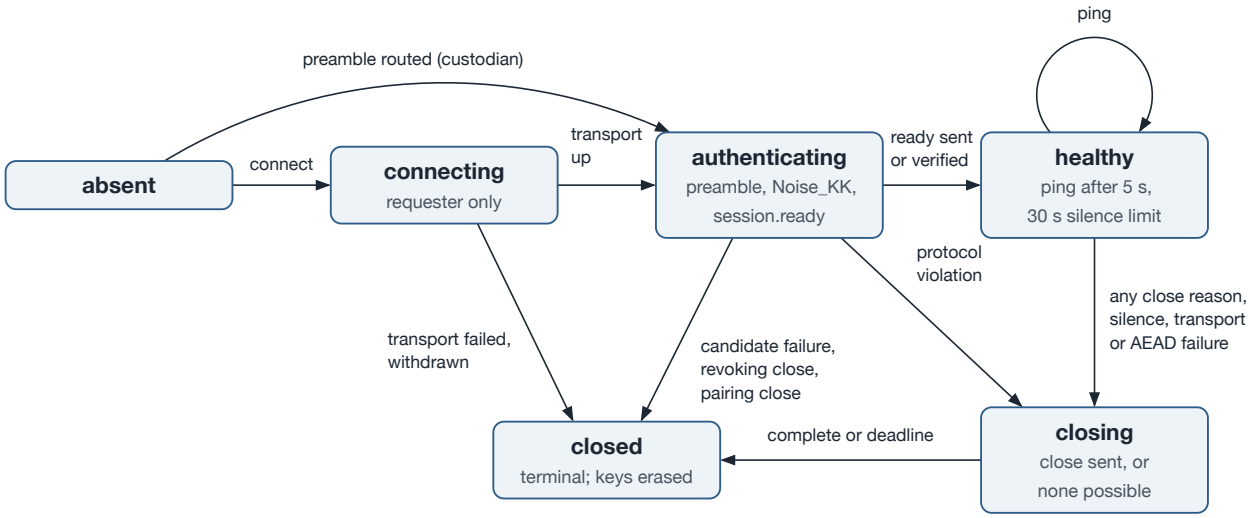

*both confirmed*: the record is stored and `pairing.stored` sent. A failed handshake also returns `handshaking` to `unpaired`.
*close sent or echoed*: a custodian once its close was sent and the wait for the echo ended, a requester on the echo alone.
*time bound*: on a custodian `revocation_notice_days` after the forget, on a requester `pairing_idle_expiry_days` after the latest session. A requester's forget in session goes to `revoking` too, with the close sent at once. *idle expiry* leads to `revoking` on a custodian only; a requester's goes straight to `revoked`, silently. A peer's `pairing_revoked` moves any paired state to `revoked`.

# 9. Session, close reasons and the rules between machines

## 9.1 Session

The `session` machine. Every session carries both endpoints' current display names (INV-21): `session.ready` carries the custodian's, and the requester's first message after it verifies `session.ready` is `session.hello` with its own, unless it is revoking, when its first message is the `pairing_revoked` close instead. On `ready_verified` the requester runs `record_peer_display_name`, `send_session_hello`, `start_liveness` and `show_connected`; the custodian takes `session_hello_received` in `healthy` and runs `record_peer_display_name`. No `operation.request` is admitted before `session.hello`, and any other first message from the requester but a revocation close is a protocol violation (X-11).

Every close is acknowledged (RAPP §7.1 item 7, X-12): an endpoint that receives an authenticated `session.close`, whatever its reason, answers with a `session.close` carrying the same reason (`echo_peer_close`) and closes the transport. An endpoint that sent a close waits in `closing` for that echo, sends nothing more and ignores every other message; a close arriving then is the echo, or a close that crossed its own, and is never answered. The wait is at most `quit_withdrawal_bound_ms`. The echo is the evidence that the peer acted on the close, such as forgetting the pairing; without it the close is not known to have been delivered: a requester keeps the forgotten pairing in its revocation backlog for the next session, and idle expiry and the loss handling apply as before (UCA-124, CC-123).




## 9.2 What a received close does

Each received close below is answered with its echo before the transport closes.

| Reason received | Requester: view of the custodian | Requester: pairing | Custodian: pairing and list |
|---|---|---|---|
| `pairing_revoked` | `ended`; "the phone forgot this computer" | revoked, key destroyed | `revoked`, not listed |
| `service_withdrawn` | `withdrawn`; identity withdrawn at once | kept | `paired_disconnected`, shown off |
| `card_unavailable` | `withdrawn`; identity withdrawn at once, no grace, no redial | kept | (the custodian only sends it) |
| `protocol_violation` | `idle`, no redial | kept | `paired_disconnected` |
| `normal`, `complete`, `shutdown`, `user_disconnect`, `policy` | a loss: `connecting` | kept | `paired_disconnected` |
| none: transport loss, silence, AEAD or framing failure | a loss: `connecting` | kept | `paired_disconnected` |

## 9.3 Cross-component rules

| ID | When | Require |
|---|---|---|
| X-01 | a pairing leaves `paired_connected` with a live session | the session receives `close_requested_by_pairing` |
| X-02 | a session enters `closing` | its operation receives `session_closed` |
| X-03 | an operation is admitted | the pairing is `paired_connected`, the session `healthy`, and no other operation is live on it |
| X-04 | an operation needs the card | one operation holds the card across every session, in arrival order |
| X-05 | card presence or reader attachment changes | every healthy session receives `card.status` |

| ID | When | Require |
|---|---|---|
| X-06 | a requester receives `service_withdrawn` or matches a withdrawn record | its view enters `withdrawn` and dials, offers and holds nothing until `record_appeared` or `advertisement_appeared` |
| X-07 | a requester dials | the dial follows an appearance, a consumer request, or a `redial_due` within `redial_window_ms` |
| X-08 | a custodian's holder turns remote access off | every pairing is untouched and the withdrawal runs |
| X-09 | a requester's pairing enters `revoking` | one dial each time its custodian appears and none on a timer, until `revocation_notice_sent`, `peer_revocation_notice` or `notice_window_expired` |
| X-10 | a requester's pairing becomes active | every other pairing it holds receives `forget_pairing` |
| X-11 | a custodian's session is `healthy` | no `operation.request` is admitted before `session.hello`, and any other first message but a revocation close is a protocol violation |
| X-12 | an endpoint receives an authenticated `session.close` | it answers with a `session.close` carrying the same reason and closes the transport, unless it waits for the echo of its own close, which the received close then is |

## 9.4 Lifecycle invariants

| ID | Statement |
|---|---|
| INV-14 | No withdrawal revokes a pairing. |
| INV-15 | No pairing expires while a session to it is established. |
| INV-16 | No dial and no offer while the requester's view of the custodian is `withdrawn`. |
| INV-17 | A requester holds at most one active pairing. |
| INV-18 | A requester keeps a revoking pairing until its custodian echoes the close or the pairing's idle expiry would have passed. |
| INV-19 | No pairing is found, matched or replaced by its display name. |
| INV-20 | The display name lives in the pair record and dies with it. |
| INV-21 | Every session carries both endpoints' current display names. |

# 10. Roles and names

The custodian is a phone that reads the card over NFC: an iPhone or an Android phone. A requester is a computer (macOS, Windows, Linux or BSD), an iPad or an Android tablet; a tablet uses a card in a reader attached to it or a phone's card over RAPP and never serves a card. The requester rules of this document, the switch that forgets and its revocation backlog included, apply to tablets as to computers.

Each endpoint sends its display name in `pairing.hello`, and again on every session: the custodian in `session.ready`, the requester in `session.hello`. It is required and not blank (RAPP §4.3 item 3, §7.1). A name that already says the platform carries no operating-system prefix; the prefix `<operating system> –` appears only where the name does not reveal the platform:

| Device | Name |
|---|---|
| Mac | `macOS – example-host` |
| Windows computer | `Windows – EXAMPLE-HOST` |
| Linux or BSD computer | `Linux – example-host` |
| Android tablet | `Android – Example Tablet` |
| iPhone | its model name until the user-assigned name is available, such as `iPhone 16` |
| iPad | its model name until the user-assigned name is available, such as `iPad Pro 11-inch (M4)` |
| Android phone | its device name, without a prefix |

The peer stores the name inside the pair record, so record and name live and die together (INV-20). On every session each side checks the name it receives as at pairing and replaces the stored name when it differs, so a renamed device shows its new name from its next session on; a store that fails keeps the old name and the session goes on. A name is a label for people and identifies nothing: no endpoint finds, matches, replaces or revokes a pairing by its name (INV-19).

# 11. Known limits

- **No stable requester identity.** Every pairing ceremony makes fresh keys, so a custodian cannot tell that a new pairing comes from a computer it knew before. A pairing the requester forgot leaves the custodian's list when the requester's revocation close reaches it, in session or by its one bounded attempt, and otherwise by the custodian's idle expiry after 30 days without a session; until then the custodian lists both.
- **Ends that send nothing.** An endpoint that crashes, is killed by its operating system or loses power sends nothing, and neither does a custodian that gets no quit signal. Its peer handles that as a loss within the bounded windows: the session silence limit, `redial_window_ms` and the identity grace (RAPP §4.6 item 8). This limit is kept.

# 12. Analysis

The STPA of these machines is in the RAPP STPA record, with unsafe control actions UCA-110 to UCA-124 and controller constraints CC-113 to CC-123.